

DEPARTMENT OF SOFTWARE ENGINEERING
BACHELORS IN SOFTWARE ENGINEERING
Course Code: SE-408
Course Title: Design Patterns
Complex Engineering Problem
BE Batch 2023, Fall Semester 2025

Course Learning Outcome

CLO2: Apply understanding of operating system design and its impact on application design and performance. (C3-PLO2)

Complex problem-solving attributes (CPA) covered (as per PEC - OBA manual – 2019)

- **CPA-1 Depth of analysis required:** These problems do not have readily apparent solutions and demand abstract thinking and innovative analysis to develop appropriate models.
- **CPA-2 Level of interaction:** Necessitate the resolution of substantial challenges resulting from the interplay of diverse or conflicting technical, engineering, or other factors.
- **CPA-3 Familiarity:** Can go beyond prior experiences through the application of principle-driven methodologies.

Problem Statement

Students are required to analyze and modify the xv6 operating system's process scheduler. Students will research various scheduling algorithms, select and implement a suitable one, and analyze its direct impact on the performance of I/O-bound and CPU-bound applications, justifying their choice of scheduler based on empirical evidence.

Activities:

1. OS Initialization and Setup

- **Environment Setup:** Configure the xv6 development environment using the QEMU emulator and the RISC-V toolchain.
- **Code Familiarization:** Study the existing xv6 source code for the scheduler (kernel/proc.c), system call interface, and process management.
- **Baseline Compilation:** Compile and run the unmodified xv6 to ensure the environment is working correctly and to establish a performance baseline with the default round-robin scheduler.

2. Kernel Modification: Scheduler Selection and Implementation

- **Research and Select a Scheduler:** Investigate various scheduling algorithms beyond the default round-robin scheduler. Examples include, but are not limited to:
 - Multi-Level Feedback Queue (MLFQ)
 - Preemptive Priority Scheduling
 - Shortest Remaining Time First (SRTF) Select **one** scheduler for implementation. Your choice should be based on a clear hypothesis about how it will perform with mixed workloads.
- **Implement the Chosen Scheduler:** Replace xv6's default round-robin scheduler with the algorithm you selected. You will need to modify the kernel code to implement the logic of your chosen algorithm.
- **System Call for Performance Analysis:** Add a new system call, `getprocinfo(int pid)`, that returns performance metrics for a specific process, such as total CPU ticks consumed and the number of times it has been scheduled.

3. Application Development for Performance Analysis

- **Develop a CPU-Bound Application:** Write a user-level program that performs an intensive computational task (e.g., calculating a large number of prime numbers) to simulate a CPU-bound workload.
- **Develop an I/O-Bound Application:** Write a user-level program that frequently performs I/O operations (e.g., repeatedly writing small amounts of data to a file) to simulate an I/O-bound workload.

4. Testing and Performance Analysis

- **Functional Testing:** Verify that your new scheduler correctly manages processes and that the `getprocinfo` system call returns accurate data.
- **Benchmark Execution:** Run both the CPU-bound and I/O-bound applications simultaneously on both the original round-robin scheduler and your **newly implemented scheduler**.
- **Performance Report:** Write a comprehensive technical report analyzing your findings. The report **must**:
 1. **Justify** your choice of scheduling algorithm with a clear rationale and hypothesis.
 2. **Compare** the performance (e.g., completion time, CPU ticks) of both applications under each scheduler. Use graphs and tables to present your data.
 3. **Explain why your chosen scheduler** impacts the performance of CPU-bound and I/O-bound tasks differently compared to the round-robin scheduler.
 4. **Conclude** with a clear analysis of how this specific OS design choice directly influences application design and performance, and reflect on whether your initial hypothesis was correct.

Deliverables:

You are required to submit a single compressed file (.zip or .tar.gz) containing the following items. The submission should be made via Google Classroom by the specified deadline.

1. Technical Report (PDF Format):

- A comprehensive report detailing your work, structured according to the sections outlined in **Activity 4**. This report is the primary document for assessment and must include your analysis, design, implementation details, and performance evaluation.
- The provided assessment rubric must be attached as the first page of your report, with your name and roll number filled in.

2. Source Code:

- **Modified Kernel Files:** All xv6 kernel files that you have modified (e.g., `kernel/proc.c`, `kernel/proc.h`, `kernel/trap.c`, `kernel/syscall.c`, etc.).
- **User-Level Applications:** The source code for your two benchmarking applications (`cpubound.c` and `iobound.c`).
- **Patch File:** A single patch file (`cep_solution.patch`) generated using `git diff`. This file should encapsulate all changes made to the original xv6 source code.

3. Demonstration Video (Optional but Recommended):

- A short (2-4 minute) screen recording demonstrating your project. The video should clearly show:
 - Running the benchmark applications on the **original** xv6 scheduler.

- Running the benchmark applications on your **new MLFQ** scheduler.
- A brief explanation of the output, highlighting the performance difference.

You are required to bring a hard copy of the report for assessment that will be conducted in week 12.

While compiling the report, keep the following in mind:

- o **Attach the provided rubric on top of your report, with names and roll numbers filled in.**
- o **Title page, along with rubric sheet.**
- o **Use font size of 12 and/or 14 for headings and 11 for regular text.**
- o **The font style should be Times New Roman.**